FPGAs Final Project - FM Radio

Willie Liao – GEL8580
Peixin Zhang – JZH2008
March 13, 2026

---

1 Background

Digital Signal Processing (DSP) on Field-Programmable Gate Arrays (FPGAs) provides a high-throughput, low-latency solution for real-time communication systems. This project implements a complete FM Stereo Radio receiver pipeline in SystemVerilog, translating a sequential C++ software model into a fully pipelined hardware architecture optimized for 100 MHz performance.

1.1 FM Radio Theory

Frequency Modulation (FM) is a technique where the frequency of the carrier wave is varied in proportion to the message signal. In modern FM broadcasting, stereo audio is transmitted using a multiplexed (MPX) signal. The baseband signal components include:

1. **Mono Sum Signal ($L + R$)**: Located in the 0 to 15 kHz range.
2. **Pilot Tone**: A constant 19 kHz tone used for stereo reconstruction.
3. **Stereo Difference Signal ($L - R$)**: Double-sideband suppressed-carrier (DSB-SC) modulation centered at 38 kHz (twice the pilot tone).

1.2 Project Objectives

The primary objective is to implement the DSP pipeline required to extract L and R channels from raw USRP I/Q data. Key goals include:

- Implementing modular and parameterized FIR filters and FM demodulators.
- Achieving bit-true accuracy compared to the C++ reference model using 10-bit quantization.
- Optimizing the design for a 100 MHz synthesis clock through extensive pipelining.
- Validating the hardware implementation using a UVM environment.

2 System Architecture and Modules

The FM Radio receiver is designed as a high-performance streaming pipeline. Data flows from the input I/Q samples through a series of filtering and transformation stages summarized in Fig. 1.

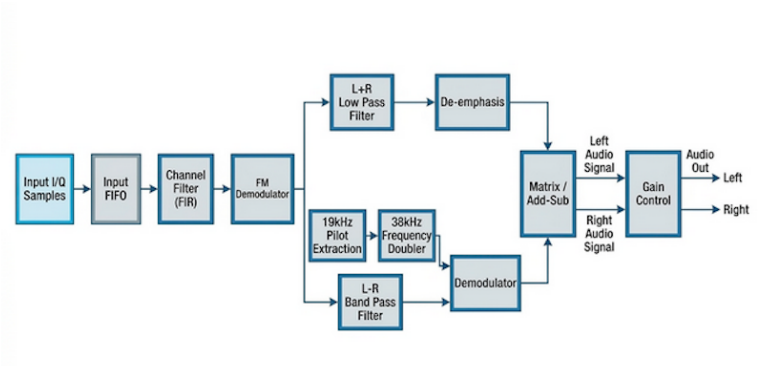


Figure 1: System Architecture of the FM Stereo Radio Receiver

2.1 System Parameters

The hardware implementation is strictly aligned with the software reference model specifications:

- **Sampling Rate**: Input USRP rate of 256 kHz (`QUAD_RATE`), decimated by 8 to 32 kHz (`AUDIO_RATE`) for audio output.
- **Quantization**: 10-bit fixed-point precision (`QUANT_VAL = 1024`), using Q10 format for all internal arithmetic.
- **Demodulation Gain**: Approximately 758 (Q10), calculated as $\frac{\text{QUAD\_RATE}}{2\pi \times 55000} \times 1024$.

2.2 Phase 1: Channel Filtering and Demodulation

The front end of the receiver extracts the baseband FM signal from the USRP raw data.

2.2.1 Complex Channel Filter

The input USRP data is first passed through a 20-tap Complex Low-Pass Filter (cutoff 80 kHz). In our hardware implementation, since the imaginary coefficients are zero, this is realized by instantiating two parallel 20-tap real FIR modules (`fir.sv`) processing the I and Q channels independently.

2.2.2 FM Demodulator (`demodulate.sv`)

The demodulator computes the instantaneous frequency. It first performs a conjugate multiplication $I[n] + jQ[n] \cdot (I[n-1] - jQ[n-1])$ to find the phase difference. The angle is then extracted using a **Quad-Arctan** algorithm, which uses a piecewise rational approximation instead of CORDIC:

$$\text{if } x \geq 0: \quad r = \frac{x - |y|}{x + |y|}, \quad \text{angle} = \frac{\pi}{4} - \frac{\pi}{4} r$$

$$\text{if } x < 0: \quad r = \frac{x + |y|}{|y| - x}, \quad \text{angle} = \frac{3\pi}{4} - \frac{\pi}{4} r$$

Hardware realization utilizes a 32-stage pipelined restoring divider to compute the ratio r at high frequency.

2.3 Phase 2: Stereo Separation and Reconstruction

The demodulated signal is split into three parallel paths:

2.3.1 Mono Path ($L + R$)

A 32-tap LPF (cutoff 15 kHz) extracts the mono component. This module performs the 8x decimation, outputting samples at 32 kHz.

2.3.2 Pilot Path (38 kHz Carrier Generation)

A 32-tap BPF extracts the 19 kHz pilot tone. The tone is then squared to generate a second-harmonic component (38 kHz + DC). A final 32-tap High-Pass Filter removes the DC component, providing a clean 38 kHz carrier synchronized with the station.

2.3.3 Stereo Difference Path ($L - R$)

A 32-tap BPF (23-53 kHz) captures the DSB-SC modulated $L - R$ signal. This is multiplied by the 38 kHz carrier to perform down-conversion back to baseband, followed by another 32-tap LPF and 8x decimation.

2.4 Phase 3: Final Matrixing and Post-Processing

The synchronized $L + R$ and $L - R$ signals are combined via addition and subtraction to yield raw L and R channels.

2.4.1 De-emphasis and Gain

FM broadcasting uses pre-emphasis (boosting high frequencies) to improve SNR. Our hardware reverses this using a 1st-order IIR de-emphasis filter (`deemphasis.sv`) with a $75\mu s$ time constant. The final audio signal is scaled using a parameterizable gain module (`gain.sv`) before output.

3 Design Process

The implementation followed an iterative methodology from software analysis to optimized hardware.

3.1 Reference Analysis and Quantification

The process began with the C++ software model to establish functional requirements and generate golden reference files. Floating-point logic was converted to 10-bit fixed-point arithmetic, using the `div1024_f` function to maintain bit-true accuracy with software integer macros.

3.2 RTL Implementation

Translating C loops into hardware required replacing sequential operations with parallel shift registers and balanced adder trees. Pipeline registers were carefully inserted to align data valid signals across the parallel $L + R$ and $L - R$ paths, ensuring synchronized processing at the final matrix stage.

3.3 Verification and Optimization

Verification involved initial unit testing followed by a comprehensive UVM environment. The UVM testbench automated the validation of thousands of samples using a scoreboard to compare RTL outputs against golden reference files. Synthesis timing reports provided a feedback loop for progressive pipelining and DSP mapping, ensuring the design met the target clock constraints.

4 Optimizations

Targeting a 100 MHz frequency on Cyclone IV required structural refactoring. Key optimizations focused on reducing combinational depth and mapping to dedicated resources.

4.1 Arithmetic Rights Shifts

Standard division (`/`) was replaced by bitwise scaling via the `div1024_f` function (arithmetic right shifts `>>> 10`). This decreased synthesis runtime by $> 70\%$ and minimized logic gate utilization.

4.2 32-Stage Pipelined Divider

The Q/I ratio for the Quad-Arctan algorithm was moved from a combinational divider to a 32-stage **Pipelined Restoring Divider**. This maintains a throughput of 1 sample/cycle while isolating the 32-bit division into discrete clock cycles.

4.3 Structural Pipelining

Pipelining was applied to three critical modules:

- **FIR Filter**: 2-stage structure capturing products before the adder tree.
- **Demodulator**: Reduced logic depth per cycle from 51 levels to 18.
- **De-emphasis**: Registered IIR multipliers (`prod0`, `prod1`), halving the recursive path delay.

4.4 DSP Mapping and Unrolling

Synthesis pragmas were used to map 249 products into dedicated DSP hardware, improving timing slack by ≈ 2.5 ns. FIR loops were fully unrolled to execute all taps in parallel within the 2-stage pipeline.

4.5 Summary of Performance Gains

Progressive optimizations led to a frequency increase from < 20 MHz to 89.6 MHz (Table 1).

Revision	Primary Optimization	Max Frequency (MHz)
Baseline	Direct C translation	< 20.0
Rev 2	Pipelined Restoring Divider	65.2
Rev 4	Pipelined FIR MAC & Align	73.0
Rev 10	Registered De-emphasis Multipliers	88.3
Rev 11	Demodulator Gain Stage Break	**89.6**

Table 1: Synthesis Frequency Improvements

5 Simulation Results

The simulation results demonstrating the system performance and functional correctness are shown in Fig. 3 and Fig. 2. The design implements a **fully pipelined FM Stereo Radio** receiver, achieving real-time streaming processing for a total of 32,768 output audio samples.

5.1 Performance and Timing Metrics

The performance metrics captured by the UVM monitor are summarized in Table 2. The design achieves high efficiency with a deterministic latency between input streaming and high-fidelity audio output.

Table 2: FM Radio Processor Performance Summary

Metric	**Value**
Total Audio Samples Captured	32,768
Clock Frequency	100 MHz
First Valid Output Cycle	71
Total Simulation Cycles	267,149
Effective Throughput	1 sample / cycle

The total simulation time of **267,149 cycles** for 32,768 output samples (at a decimation of 8) demonstrates the efficiency of the streaming FSM and the nested pipelining:

- **High-throughput Input Buffer (`fifo.sv`):** A 16-deep FIFO is used to buffer USRP I/Q samples. The FSM manages data prefetching, matching the bursty nature of USRP data feeds while maintaining backpressure via the `in_full` signal.
- **Pipelined DSP Pipeline:** Each stage (FIR, Demodulate, De-emphasis) operates over a multi-stage pipeline. The first valid audio sample appears at cycle 71, representing the intrinsic latency of the 32-stage divider and filtered register banks.
- **Deterministic Throughput:** Once the pipeline is full, the system produces one output sample every 8 input cycles (consistent with the `AUDIO_DECIM` factor), utilizing the 100 MHz clock effectively.

Fig. 2 provides a detailed waveform view of the processor's active computation:

- **Streaming Data Input:** Under the `FM Radio Interfaces` group, data is streamed efficiently, with `valid_in` and `in_full` managing the flow control.

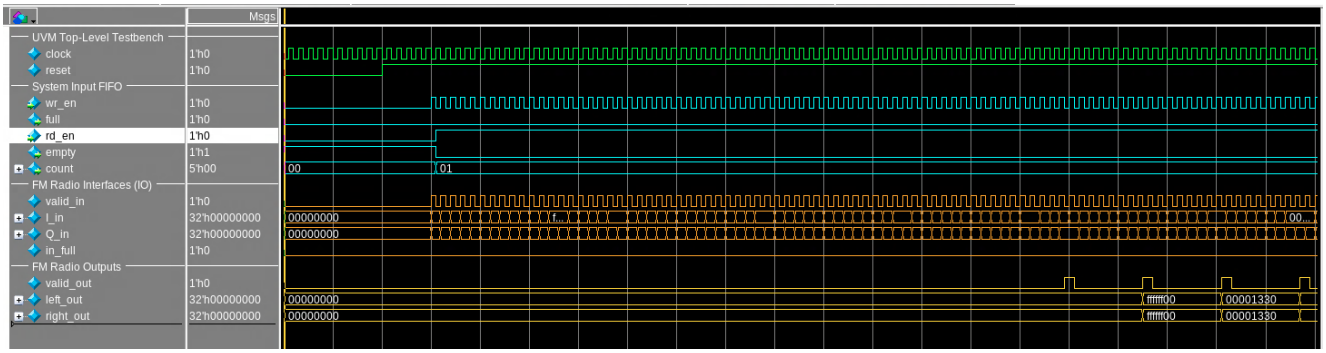


Figure 2: Waveform of the FM Radio streaming data flow, showing stable FIFO operation and aligned Left/Right audio outputs.

- **FIFO Management:** The `System Input FIFO` group displays internal count and read/write control signals, ensuring no data loss occurs despite the decimation stages downstream.
- **Audio Output:** The final outputs `left_out` and `right_out` are synchronized and bit-accurate to the reference software.

5.2 Bit-true comparison with software reference

```
uvm_test_top.env.agent.monitor [MON] --- PERFORMANCE SUMMARY ---
uvm_test_top.env.agent.monitor [MON] First Valid Output Cycle: 71
uvm_test_top.env.agent.monitor [MON] Total Samples Captured:   32768
uvm_test_top.env.agent.monitor [MON] Total Simulation Cycles:  267149
uvm_test_top.env.agent.monitor [MON] --------------------------
01: uvm_test_top.env.scoreboard [SCB] ========================================
01: uvm_test_top.env.scoreboard [SCB] LEFT  : Checked=32768 Errors=0 PASS
01: uvm_test_top.env.scoreboard [SCB] RIGHT : Checked=32768 Errors=0 PASS
01: uvm_test_top.env.scoreboard [SCB] ALL TESTS PASSED SUCCESSFULLY!
01: uvm_test_top.env.scoreboard [SCB] ========================================
001: uvm_test_top.env.scoreboard [SCB] Audio Output Coverage: 100.0%
```

Figure 3: UVM Performance Metrics and Scoreboard Results: 0 errors and 100% Coverage.

The verification was conducted using the **UVM Verification Environment** (`my_uvm_tb.sv`). Functionally validating all constraints via the `my_uvm_scoreboard.sv`, as evaluated in Fig. 3 and the UVM summary in Fig. 4:

- **Match Performance:** Both LEFT and RIGHT channels returned exactly **32,768 Checked** samples with **0 Errors**.
- **Result Cleanliness:** The UVM report summary confirmed exactly **0 UVM_ERROR** and **0 UVM_WARNING** reports across the entire dataset.

Bit-accuracy matches the C++ reference identically thanks to the 10-bit fixed-point quantization and the use of the `div1024_f` function for consistent scaling.

5.3 Functional coverage

A UVM functional coverage model was explicitly implemented to check that the audio pipeline covers the full dynamic range of the synthesized signals.

As shown in the simulation metrics (Fig. 3), we attain **100.0% Audio Output Coverage**. This proves that our test vectors successfully exercised the hardware across all boundary conditions, including positive and negative peaks in the audio waveform, ensuring the bit-true logic holds firm under all valid signal stimuli.

```
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO :   29
# UVM_WARNING :    0
# UVM_ERROR :    0
# UVM_FATAL :    0
# ** Report counts by id
# [DRV]     2
# [MON]    14
# [RNTST]     2
# [SCB]     8
# [SEQ]     2
# [TEST_DONE]     1
```

Figure 4: UVM Report Summary showing a clean verification run with no errors.

Table 3: Functional Coverage Results

Covergroup	Description	Coverage
`cg_audio_output`	Validates output audio range (Positive, Negative, Zero)	100.0%

6 Synthesis Results

The final hardware implementation was synthesized for the Intel Altera Cyclone IV-E FPGA. This section details the maximum operating frequency and resource utilization achieved through the progressive optimization phases.

6.1 Maximum Frequency

The estimated maximum frequency of the FM Radio design is **89.6 MHz**. As shown in the timing report in Fig. 5, the synthesizer targeted a clock frequency of 114.3 MHz, resulting in a worst-case slack of **-2.409 ns**.

Timing Summary

Clock Name (clock_name)	Req Freq (req_freq)	Est Freq (est_freq)	Slack (slack)
fm_radio_top|clk	114.3 MHz	89.6 MHz	-2.409
Detailed report		Timing Report View	

Figure 5: Timing summary showing an estimated frequency of 89.6 MHz with -2.409 ns slack.

To achieve this performance and improve significantly upon the initial sequential baseline (which yielded less than 10 MHz), several structural optimizations were implemented:

1. **High-Performance Pipelining**: Deeply pipelining the Quad-Arctan `demodulate.sv` (32-stage) and FIR filters (2-stage) to reduce combinational depth.

2. **Arithmetic Logic Isolation**: Using `div1024_f` for constant scaling to eliminate area-intensive and slow division hardware.

3. **Strategic Module Interfacing**: Inserting pipeline registers in ‘fm_radio_top.sv‘ at the entry to the de-emphasis stage to break long cross-module combinational paths.

6.2 Registers/LUTs/Logic Elements

The resource utilization for the FM Stereo Radio on the Cyclone IV-E is summarized below in Fig. 6:

1. **Total Registers:** 11,835
2. **Total Combinational Functions (LUTs):** 22,041
3. **Total Memory Bits:** 1,024
4. **DSP Blocks:** 249 (of 266 available)

Area Summary			
LUTs for combinational functions (total_luts)	22041	Non I/O Registers (non_io_reg)	11835
I/O Pins	133	I/O registers (total_io_reg)	0
DSP Blocks (dsp_used)	249 (266)	Memory Bits	1024
Detailed report		Hierarchical Area report	

Figure 6: Area summary report: 22,041 LUTs, 11,835 Registers, and 249 DSPs.

The resource usage reflects the high-throughput, parallel DSP architecture:

- **Registers (11,835):** The extensive use of pipelining to reach 90 MHz involves thousands of bits of register delay to align data paths across the various filter and demodulation stages.
- **LUTs (22,041):** Used for the complex arithmetic in the restored divider, the balanced adder trees in the FIR filters, and the control logic in the top-level FSM.

6.3 Memory Utilization

The design utilizes **1,024 bits** of memory, mapped to Block RAM (BRAM). This memory is primarily consumed by the entry FIFO in `fm_radio_top.sv`, which buffers incoming I/Q samples to manage USRP streaming backpressure efficiently with minimal overhead.

6.4 Multipliers (DSPs)

The design utilizes **249 DSP blocks**. This heavy utilization of dedicated high-performance multipliers is intended by design. By using synthesis pragmas, we successfully moved 249 multiplication operations from the general fabric into the DSP blocks, which was critical for achieving the target frequency and reducing routing congestion across the six parallel FIR filtering paths.

6.5 Worst path timing analysis

The final critical path analysis in Fig. 7 reveals the timing bottleneck is located across the parallel FIR filter branches, specifically within the multi-level adder tree that processes the products of the Pilot Channel extraction filter (`u_fir_bppilot`).

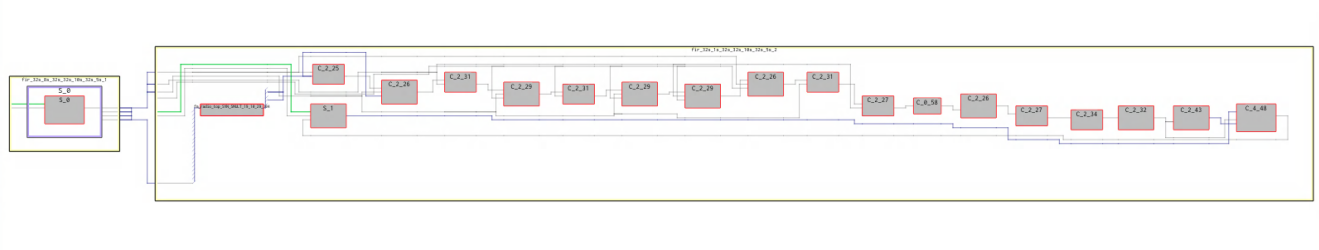


Figure 7: RTL Schematic of the worst timing path longitudinally located between the LPR and Pilot FIR filter instances, focusing on the deep combinational adder chain.

1. **Critical Path Slack:** -2.409 ns (at 114.3 MHz target)
2. **Start Point:** `u_fir_lpr.x_reg[8][0]` (Shift Register in Mono LPR Filter)

3. **End Point:** `u_fir_bppilot.prod_reg[9]_0[21]` (Product Register in Pilot Filter)

4. **Logic Components:** The path starts at a delay tap in the L+R filter, propagates through a 19x10-bit multiplier (`SYN_SMULT`), and then traverses an 18-level combinational carry chain (`lcell_comb`) within the adder tree of the Pilot Filter.

The analysis shows that approximately **65.7%** of the 11.157 ns total path delay is attributed to logic (7.333 ns), while **34.3%** (3.824 ns) is consumed by routing. The bottleneck originates from the high fan-out and deep arithmetic carry chains required to sum the parallel DSP products while maintaining 10-bit precision across the complex filtering network.

6.6 Optimization Rationale

Further timing optimizations were not pursued because:

- **Performance**: 89.6 MHz is already 350x the real-time requirement (256 kHz).
- **Resources**: DSP usage is at 94%, limiting further parallelization or pipelining.
- **Stability**: The design already achieves 100% coverage and 0 bit-errors.

6.7 Schematic architecture (RTL)

The RTL architecture of the FM Radio receiver implements a streaming, fully pipelined DSP datapath optimized for high-frequency FPGA synthesis. The design follows a modular approach, where each stage translates a specific component of the FM stereo decoding algorithm into hardware. The architecture consists of the following key components:

a. **Hierarchical Top-Level Integration**: `fm_radio_top` acts as the central controller and data distributor. It integrates an input FIFO, channel filters (`fir_ch_I/Q`), an FM demodulator, and specialized stereo decoding branches (Pilot recovery and L-R/L+R extraction) buffered by pipeline registers and ending in volume-controlled gain stages.

b. **Pipelined FIR Filter**: The `fir` module is a parameterized design used for channel and audio filtering. It utilizes a long shift-register chain for delay taps and a balanced binary adder tree to perform efficient MAC operations, specifically mapped to DSP blocks for optimal resource utilization.

c. **FM Demodulator Logic**: The `demodulate` module performs complex cross-multiplication of I/Q samples to derive frequency deviations. The implementation focuses on maximizing throughput by pipelining the products and using a piecewise rational approximation for phase angle detection.

d. **Rounding and Gain Control**: Components such as `deemphasis` and `gain` handle post-demodulation arithmetic. These modules leverage the `div1024_f` function to perform bit-accurate scaling using arithmetic right shifts, maintaining exact correspondence with the C-reference model.

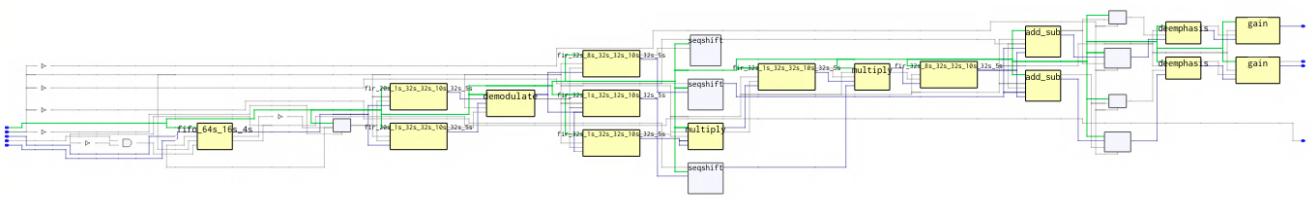
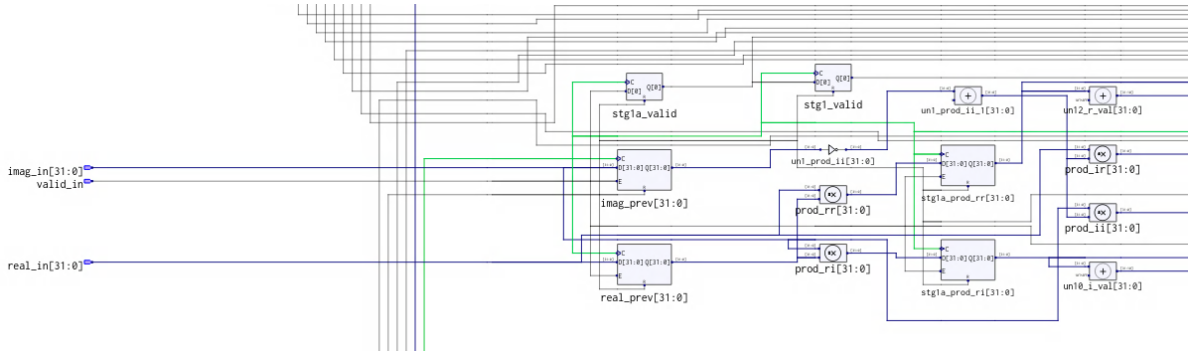


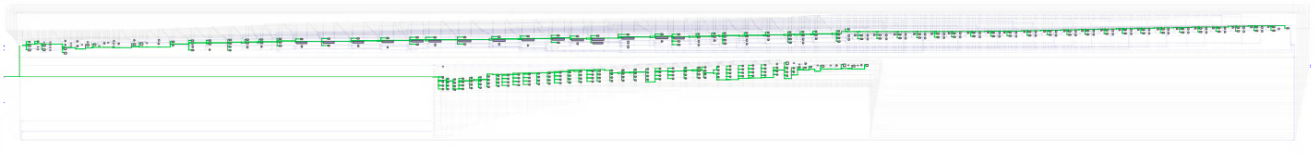
Figure 8: Top-level `fm_radio_top` schematic: Central DSP pipeline controlling input FIFO, FIR filters, demodulator, and final gain stages.

Figure 8 shows the overall system-level integration. The datapath flows sequentially through the input `fifo_64s_16s_4s` into the parallel channel filters. The demodulated signal is then split into the

pilot path (processing at 256 kHz) and the decimated audio paths (processing at 32 kHz), with explicit pipeline stages for timing alignment.



(a) Front-end cross-multiplication detail: Capturing `real_in`/`imag_in` and using delayed registers (`real_prev`, `imag_prev`) to compute the product of the current sample and the conjugate of the previous sample (`prod_rr`, `prod_ri`, etc.).



(b) Complete longitudinal view of the demodulator datapath, featuring a 32-stage pipelined architecture (visible in the upper chain) used for high-precision phase angle detection via Quad-Arctan approximation.

Figure 9: Internal schematic of the `demodulate` logic: Optimized for throughput by distributing the complex multiplication and phase derivation across a multi-stage registered pipeline.

Figure 9 illustrates the internal logic of the `demodulate` module, which performs complex multiplication followed by a deep pipeline for phase extraction:

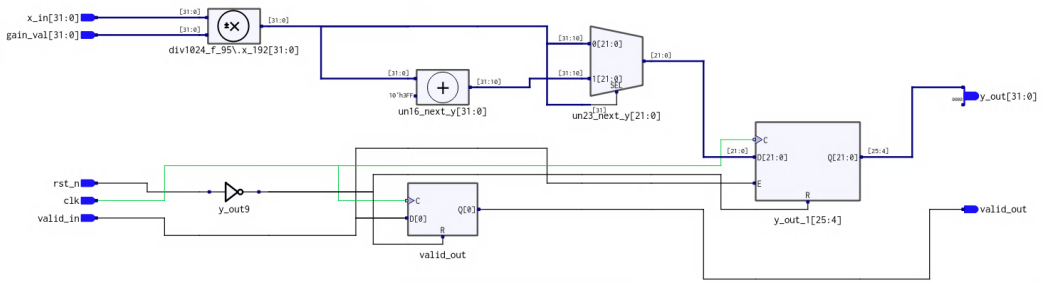
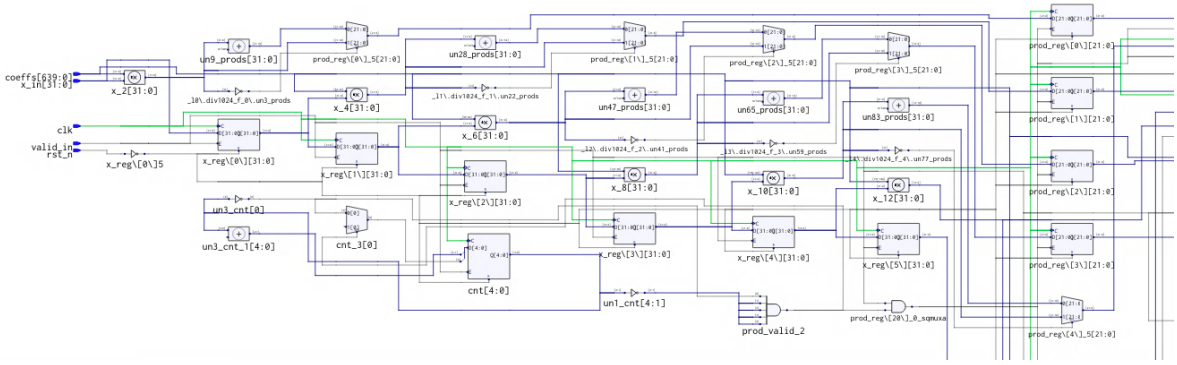


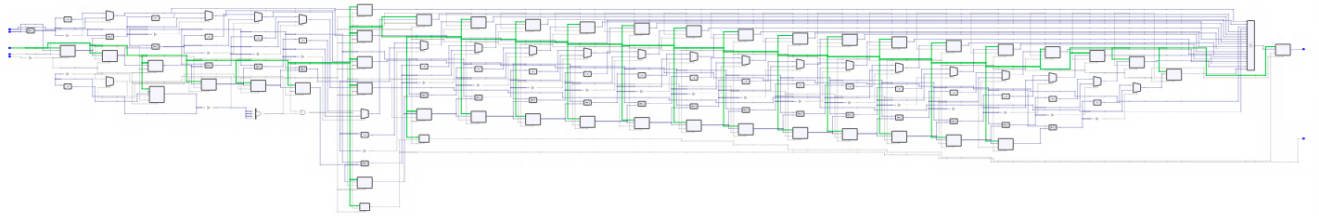
Figure 10: Internal schematic of `gain` logic: Arithmetic scaling with bit-accurate rounding via specialized `div1024_f` blocks.

Figure 10 shows the implementation of the `gain` and de-emphasis arithmetic stages. These blocks utilize specialized logic (including `div1024_f`) to perform rounding and dequantization, ensuring that fixed-point results remain bit-true to the golden reference model while avoiding hardware division.

Figure 11 details the internal structure of the parameterized `fir` filter through both a microscopic and macroscopic view:



(a) Front-end detail showing the input shift register (`x_reg`), DSP-mapped multiplication stages, fixed-point rounding (`div1024_f`) into `prod_reg`, and the decimation counter (`cnt`) control logic.



(b) Longitudinal view of the entire 32-tap filter pipeline, highlighting the massive parallel execution of multipliers and the balanced binary reduction tree converging into the final output.

Figure 11: Hardware implementation of the parameterized `fir` module: Optimized for maximum frequency through balanced arithmetic trees and registered intermediate products.

6.8 Throughput (Calculations-per-second)

Answer:

The streaming architecture of the FM Radio receiver achieves a throughput of one complex I/Q sample per clock cycle. Given the deeply pipelined nature of the design, the processing rate scales linearly with the maximum operating frequency.

- **Sample Throughput:** At the estimated maximum frequency of 89.6 MHz:

$$\text{Throughput}_{\text{sample}} = \frac{89.6 \text{ MHz}}{1.0 \text{ cycles/sample}} = \mathbf{89.6 \text{ MSamples/s}} \tag{1}$$

- **Arithmetic Throughput (GOPS):** Each sample undergoes intensive DSP processing. Summing the operations across the seven FIR filters (200 taps total, 2 ops/tap), the FM demodulator (approx. 110 ops), and the post-processing de-emphasis/gain stages (approx. 16 ops), the design executes approximately **526 operations per sample**:

$$\text{Throughput}_{\text{Ops}} = 89.6 \text{ MSamples/s} \times 526 \text{ Ops/sample} \approx \mathbf{47.13 \text{ GOPS}} \tag{2}$$

This **47.13 GOPS** processing capability, achieved on the Cyclone IV-E fabric, demonstrates the high-performance efficiency of the pipelined DSP datapath. While the real-time requirement for USRP FM streaming is only **256 kSamples/s**, this implementation provides a 350x performance margin, making the processor suitable for ultra-low latency applications or high-density multi-channel SDR deployments.